

Projet de programmation C++

Pixel Tracer

Halim Djerroud, Fabien Calcado

Table des matières

1. Le dessin vectoriel	5
1.1. Principe de base	5
1.2. Exemples d'interface de logiciel de dessin vectoriel	6
2. Objectifs du projet	8
2.1. Représentation objet des données	9
2.2. Interface en ligne de commande	9
2.3. Génération du dessin à l'écran	10
 I. Classes de base des formes	 12
3. Classe Point	12
4. Classe abstraite Shape	14
4.1. Gestion des identifiants uniques	15
5. Classe Line	15
6. Classe Circle	17
7. Classe Square	18
8. Classe Rectangle	19
9. Classe Polygon	20
 II. Affichage de l'image à l'écran	 23
10. Classes pour l'affichage	23
10.1. Classe Pixel	23
10.2. Classe Area	24
11. Tracer les formes	28
11.1. Tracer un point	28
11.2. Tracer une ligne	29
11.3. Tracer un cercle	31
11.4. Tracer carré, rectangle et polygone	33
 III. Gestion des commandes	 36
12. Les commandes utilisateur	36
13. Classe Command	36
14. Les conteneurs STL	41
15. Smart Pointers	42
15.1. <code>std::unique_ptr</code>	42

15.2. <code>std::shared_ptr</code>	42
15.3. <code>std::weak_ptr</code>	42
16. Compilation et Makefile	42

Présentation du projet

Préambule

Une image numérique est un fichier informatique permettant de stocker une image dans la mémoire de l'ordinateur (sous forme binaire). La création de cette image numérique peut être effectuée à partir de la saisie d'une image réelle par l'intermédiaire de dispositifs matériels (appareil photo numérique, caméra numérique, scanner, ...) ou produite complètement par un ordinateur, on parle alors d'image de synthèse. Une fois l'image représentée sous forme binaire, il est possible d'effectuer des traitements (modifications, transformations, filtres, ...) sur l'image grâce à des logiciels graphiques.

Comme souvent en informatique, la manière choisie pour représenter une donnée informatique va conduire à obtenir certains avantages mais aussi des inconvénients en fonction du contexte d'utilisation. Il existe deux types de représentation pour les images numériques :

- **Image matricielle** : les données de l'image sont représentées sous la forme d'une matrice de points à plusieurs dimensions. Pour des images à 2 dimensions les points sont appelés **pixels**.

Avantages : possibilité de modifier l'image pixel par pixel ce qui permet des nuances de couleurs importantes (dégradés, ombres, ...) et d'avoir des effets de textures.

Inconvénients : fichier lourd (même compressé), perte de qualité lors d'un agrandissement (visuel flou aussi appelé pixélisation de l'image). Une illustration du problème de pixélisation est donné en figure 1.

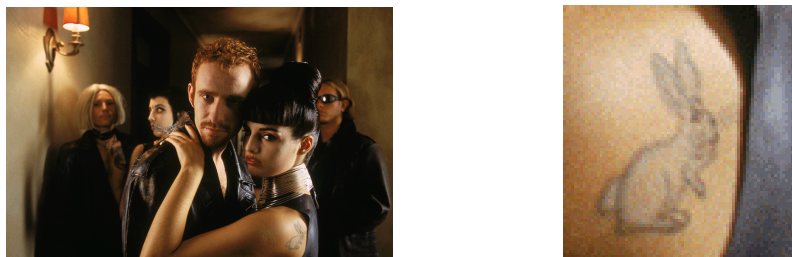


FIGURE 1 – Problème de pixélisation lors d'un agrandissement d'une zone de l'image matricielle. Image extraite du film *The Matrix* réalisé par Les Wachowski.

- Exemples de formats de fichiers matriciels : jpeg, gif, png, bmp
- Exemples de logiciels pour la création d'images matricielles : Photoshop, After Effects, Gimp

- **Image vectorielle** : les données de l'image sont représentées d'un point de vue mathématique. Une image vectorielle décrit uniquement les formes géométriques de l'image avec différents attributs que l'ordinateur est chargé de tracer à l'écran.

Avantages : fichier léger, aucune de perte de qualité lors d'un redimensionnement (pas d'effet de pixélisation). Un exemple d'agrandissement sur une image vectorielle est donné en figure 2.

Inconvénients : Ne permet de représenter que des formes géométriques simples (segments,

arcs, cercles, courbes, ...) donc non adapté à la photographie.



FIGURE 2 – Aucune perte de qualité lors de l'agrandissement d'une zone de l'image

- Exemples de formats de fichiers vectoriels : svg, ai, eps, pdf
- Exemples de logiciels pour la création d'images vectorielles : Illustrator, CorelDRAW, [Inkscape](#), [librecad](#), [Dia Diagram Editor](#)

Note : Il est possible de convertir les images d'un type vers l'autre.

Dans le cadre de ce projet nous allons nous intéresser à la réalisation des images vectorielles et plus particulièrement au fonctionnement des logiciels de dessin vectoriel. Le dessin vectoriel est très utilisé pour la création d'illustrations, de graphiques ou de cartes, car il offre à l'utilisateur la possibilité d'observer avec précision et sans perte de détails les zones qui l'intéressent en fonction de l'échelle souhaitée.

1. Le dessin vectoriel

1.1. Principe de base

Le principe des logiciels de dessin vectoriel est de stocker les informations minimales permettant de reconstruire une forme géométrique à dessiner.

Par exemple, pour une forme cercle, il suffit de stocker la position du centre et le rayon en mémoire. Ce n'est qu'au moment de l'affichage à l'écran que les positions des pixels à colorier sont calculées afin de faire apparaître la forme géométrique finale. Il est bien évidemment possible d'ajouter d'autres informations en mémoire comme par exemple la couleur du contour du cercle ou sa couleur de remplissage.

Dans les logiciels de dessin vectoriel l'outil plume est très important car il permet de créer des formes plus complexes comme des courbes. Les courbes sont représentées en mémoire par des [courbes de Bézier](#). Ces courbes sont polynomiales dont le degré est variable. Par exemple, une courbe de Bézier cubique est une courbe polynomiale de degré 3 et est définie par 4 points : 2 points correspondant aux extrémités de la courbe et 2 points correspondant à des points de contrôle. La position des points de contrôle permet d'ajuster la courbure.

Pour concevoir des dessins complexes, il est nécessaire de pouvoir travailler sur une partie de l'image sans affecter d'autres parties. L'image globale est décomposée en un ensemble de couches empilées les unes sur les autres. Chaque couche regroupe ainsi une partie des formes géométriques (i.e. les informations permettant sa reconstruction) de l'image suivant certains critères (type de forme, zone de l'image, etc). Il est donc possible de retoucher certaines parties de l'image de manière indépendante. C'est ce qu'on appelle en infographie les **calques**.

Une image vectorielle est composée d'une hiérarchie de plusieurs calques superposés les uns sur les autres dont chacun va regrouper un ensemble de formes géométriques ayant des propriétés communes.

L'ensemble des formes géométriques d'un calque sont elles mêmes hiérarchisées à l'intérieur du calque. Cette hiérarchisation à plusieurs niveaux est très importante car elle précise l'ordre dans lequel les calques, et donc les formes géométriques qui les composent, vont se superposer lors de l'affichage à l'écran. Ainsi, une forme géométrique située à un niveau inférieur dans cette hiérarchie peut être partiellement ou entièrement recouverte par une forme géométrique d'un niveau supérieur (appartenant au même calque ou à un calque de niveau supérieur).

L'utilisation de calques permet ainsi d'appliquer des modifications ciblées sur des groupes de formes géométriques tout en donnant la possibilité de verrouiller l'accès sur le groupe afin d'éviter des erreurs de manipulation. Le système de calques est un élément essentiel pour les logiciels de dessin vectoriel et offre de nombreuses fonctionnalités pour travailler de manière précise sur les éléments d'une image afin de produire des images de qualité.

Ce qu'il faut retenir sur les calques :

- un calque peut contenir plusieurs formes géométriques ;
- une forme géométrique ne peut appartenir qu'à un seul calque ;
- les modifications attribuées sur un calque sont effectuées sur toutes les formes géométriques du calque ;
- les modifications attribuées sur une forme géométrique sont effectuées seulement sur cette forme géométrique ;
- il est possible de déplacer une forme géométrique d'un calque à un autre ;
- si un calque est déplacé dans la hiérarchie des calques, toutes les formes géométriques le constituant sont déplacées avec celui-ci.

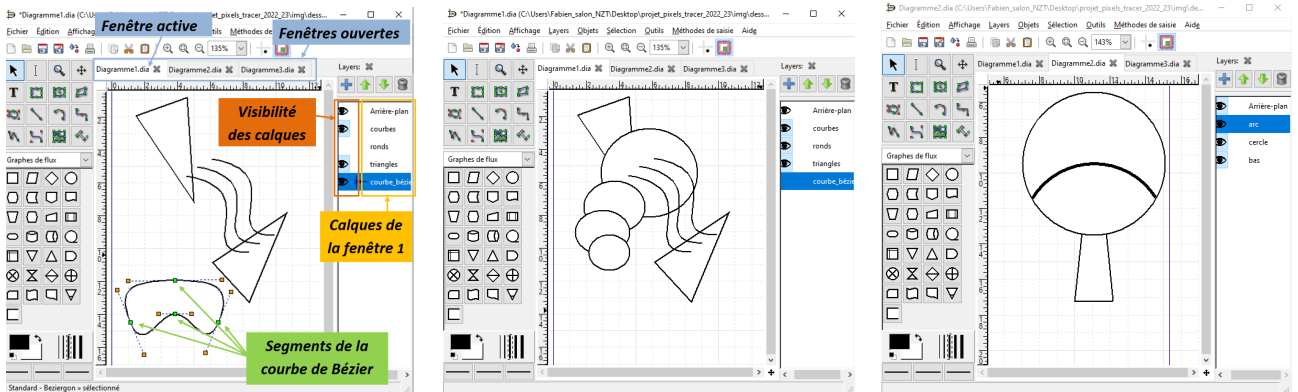
Les opérations principales sur les calques :

- créer un nouveau calque ;
- déplacer un calque dans la hiérarchie (monter ou descendre) ;
- renommer un calque ;
- supprimer un calque : entraîne la suppression des formes géométriques qu'il regroupe ;
- rendre visible ou invisible le calque : permet l'affichage à l'écran ou non des formes géométriques qu'il regroupe ;
- verrouiller ou déverrouiller le calque : autorise la modification ou non des formes géométriques qu'il regroupe.

La section suivante illustre ce système de superposition de calques à partir d'interfaces de logiciels de dessin vectoriel.

1.2. Exemples d'interface de logiciel de dessin vectoriel

Le logiciel [Dia Diagram Editor](#) (nommé Dia dans la suite du document) est logiciel de dessin vectoriel orienté pour la création de diagramme. La figure 3 montre l'interface du logiciel Dia. Ce logiciel donne la possibilité à l'utilisateur d'ouvrir plusieurs fenêtres dans une même session pour créer plusieurs images vectorielles et basculer de l'une vers l'autre facilement. Chaque fenêtre est composée d'une superposition de calques regroupant différentes formes géométriques.



(a) Contenu fenêtre 1 avec calque ronds non visible
 (b) Contenu fenêtre 1 avec calque courbe de Bézier non visible et calque ronds visible
 (c) Contenu fenêtre 2 et ses différents calques

FIGURE 3 – Logiciel Dia Diagram Editor avec son système de calques par fenêtre

Sur la figure 3a le calque **ronds** n'est pas visible donc l'ensemble des formes géométriques de ce calque ne s'affichent pas sur la zone de dessin. On peut observer sur cette même figure une courbe de Bézier assez complexe ses différents points de contrôle visibles.

Sur la figure 3b le calque **ronds** est maintenant visible tandis que le calque **courbe_bezier_fermée** ne l'est plus. Les quatre cercles regroupés dans le calque **ronds** sont donc présents dans la zone de dessin tandis que la courbe de Bézier fermée n'apparaît plus.

Sur la figure 3c c'est la fenêtre 2 qui est active. Dans cette fenêtre tous les calques sont visibles et s'affichent dans la zone de dessin qui lui est propre. Il est intéressant de remarquer le principe de superposition de calques sur cette figure. Dans la hiérarchie de calques de cette fenêtre le calque **ronds** est en dessous du calque **courbes** et au dessus du calque **triangles**. Lors du tracé dans la zone de dessin l'ensemble des quatre cercles du calque **ronds** apparaissent donc par dessus les triangles. Les trois courbes, regroupées dans le calque **courbes**, apparaissent par dessus les triangles et les cercles.

Le logiciel **Inkscape** est un logiciel de dessin vectoriel gratuit et open source. Il permet de faire des illustrations, des icônes, des logos ou diagrammes. Ce logiciel a les mêmes icônes que le logiciel Dia pour le verrouillage et la visibilité des calques. Il propose cependant la possibilité de créer des calques imbriqués comme illustré sur la figure 4. Un calque peut être composé d'un ensemble de sous-calques, chacun composé d'un ensemble de formes géométriques.

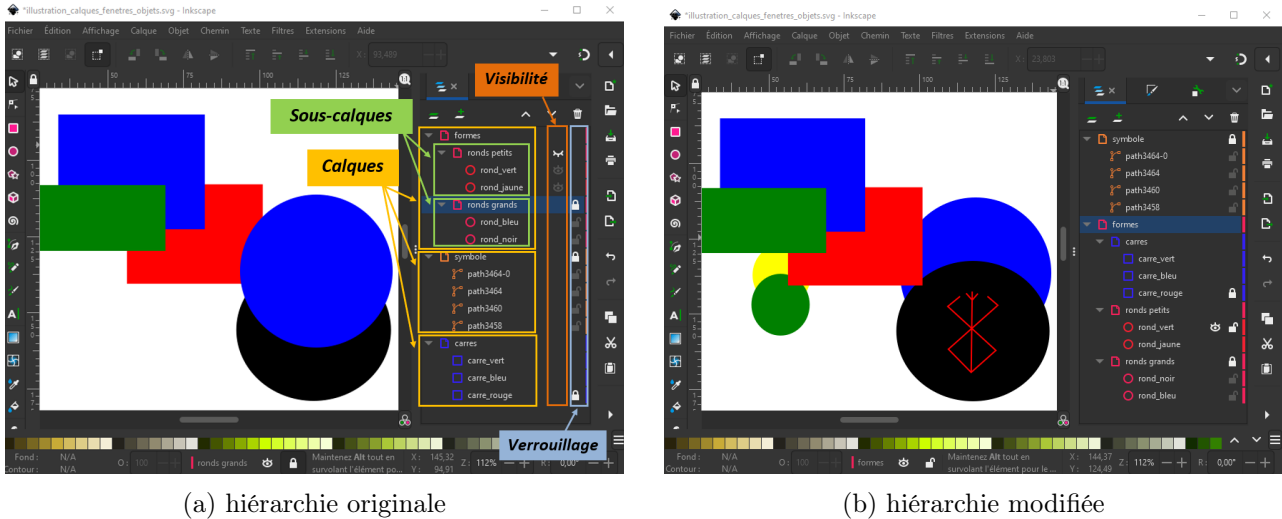


FIGURE 4 – Logiciel Inkscape avec son système de calques imbriqués

On peut remarquer sur la figure 4a que le sous-calque `ronds_petits` est non visible. Il n'apparaît donc pas sur le dessin et c'est donc la forme `rond_bleu` qui apparaît au premier plan dans la zone de dessin car il est placé le plus haut dans la hiérarchie et visible.

Sur la figure 4b la hiérarchie de calques a été modifiée par rapport à celle présente sur la figure 4a. Le calque `carres` (de couleur bleu dans zone des calques) a été intégré dans le calque `formes` (de couleur rouge). Il est donc devenu un sous-calque du calque forme et se retrouve au dessus des sous-calques `ronds_petit` et `ronds_grand` (de couleurs rouges). La forme `rond_noir` a été remontée dans la hiérarchie du sous calque `ronds_grands`. Il recouvre donc une partie du rond bleu sur le dessin dorénavant. Le calque `symbole` (de couleur orange) a été déplacé au dessus de tous les autres calques dans la hiérarchie. Ce calque regroupe un ensemble de courbes de Bézier qui apparaissent donc au premier plan sur la zone de dessin (par dessus le rond bleu et le rond noir).

2. Objectifs du projet

Dans ce projet, nous souhaitons développer une application de dessin vectoriel en mode textuel en utilisant le langage C++.

L'application à réaliser devra au minimum permettre à un utilisateur la création d'une image vectorielle simple à l'aide d'un ensemble de commandes utilisateurs et d'afficher cette image en mode textuel. L'application doit donc être capable de gérer au minimum une fenêtre et un calque regroupant toutes les formes géométriques créées par l'utilisateur.

Pour ce faire, nous devons nous intéresser à trois aspects principaux :

- la représentation objet de toutes les informations de notre application en utilisant les classes C++, l'héritage et le polymorphisme. En particulier la manière de sauvegarder les informations des formes géométriques permettant de les afficher ;
- une interface en ligne de commandes (sous la forme d'un menu) permettant à l'utilisateur d'effectuer les actions permises par l'application ;
- l'affichage à l'écran en mode textuel de l'image créée en appliquant les algorithmes de dessin décrits dans les sections correspondantes.

A partir de cette version minimale, il vous sera demandé d'implémenter diverses améliorations comme par exemple :

- la création de forme plus complexes : les courbes de Bézier ;
- une optimisation du code pour l'ajout, la suppression ou le déplacement d'une forme dans la hiérarchie (utilisation de conteneurs STL comme `std::vector` ou `std::list`) ;
- un système de calques multiples (imbriqués ou non) ;
- un système de fenêtres multiples ;
- l'utilisation de smart pointers (`std::unique_ptr`, `std::shared_ptr`) pour une gestion automatique de la mémoire.

Vous pourrez bien évidemment implémenter d'autres améliorations comme la possibilité : d'appliquer des transformations sur les formes de l'image (rotation, translation, etc), zoomer/dézoomer, de déplacer une forme d'un calque à un autre, déplacer des calques dans la hiérarchie, de sauvegarder / charger une image dans un fichier texte, de proposer des commandes permettant d'annuler ou refaire une ancienne commande (Undo / Redo).

2.1. Représentation objet des données

L'idée est de réfléchir à la représentation des formes géométriques, des fenêtres et des calques en utilisant les paradigmes de la programmation orientée objet. La figure 5 illustre une possible organisation des formes. Sur cette figure on peut constater que les données dans l'application sont stockés dans des conteneurs imbriqués. L'application possède plusieurs fenêtres (collection de fenêtres) dont chacune est composée d'une collection de calques. Chaque calque peut contenir un ensemble de formes géométriques identiques ou différentes.

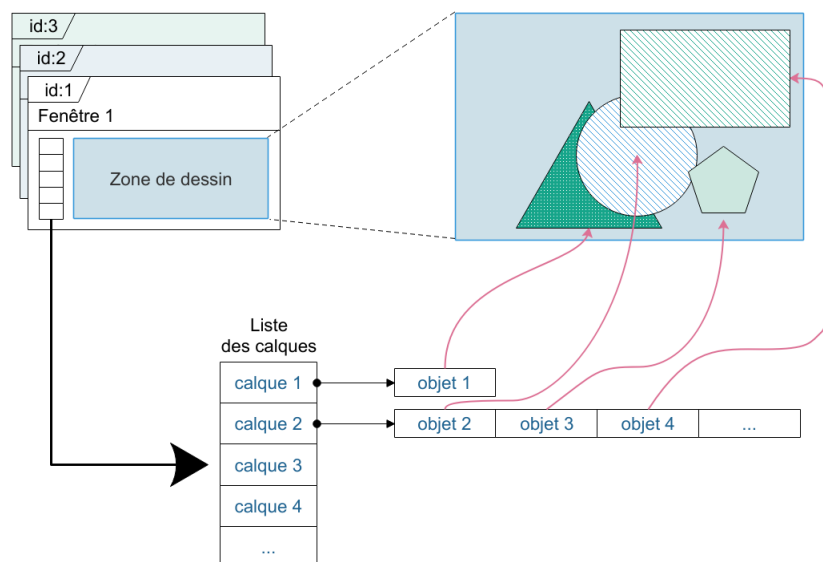


FIGURE 5 – Schéma mémoire simplifié d'une application de dessin vectoriel.

2.2. Interface en ligne de commande

Nous allons devoir créer un menu permettant à l'utilisateur d'effectuer les actions permises dans le logiciel telles que l'ajout d'une forme géométrique au calque ou l'affichage des informations des formes déjà ajoutées.

Cette interface utilisateur se présentera sous la forme d'un menu pour l'utilisateur.

Exemple de menu :

```

Veillez choisir une action :
A- Ajouter une forme
B- Afficher la liste des formes
C- Supprimer une forme
D- Tracer le dessin
E- Aide
[Autres actions]
>> Votre choix : A
    Veuillez choisir une action :
        1- Ajouter un point
        2- Ajouter une ligne
        3- Ajouter cercle
        4- Ajouter un carre
        5- Ajouter un rectangle
        6- Ajouter un polygone
        7- Revenir au menu precedent
    >> Votre choix : 2
        Saisir les informations de la ligne :
        >> Saisir le premier point x1 y1 : 12 16
        >> Saisir le deuxieme point x2 y2 : 14 18
    >> Votre choix : 3
        Saisir les informations du cercle :
        >> Saisir le point centre x1 y1 : 24 10
        >> Saisir le rayon du cercle : 7
    >> Votre choix : 7
    Veuillez choisir une action :
    A- Ajouter une forme
    B- Afficher la liste des formes
    C- Supprimer une forme
    D- Tracer le dessin
    E- Aide
    [Autres actions]
>> Votre choix : B
    Liste des formes :
        1 : CIRCLE 20 10 5
        2 : CIRCLE 20 25 5
        3 : LINE 5 20 10 25
        4 : POLYGON 15 0 5 10 10 15 5 20 10 25 5 30 15 35
        5 : CURVE 35 5 25 5 40 30 30 30
>> Votre choix : D
    
```

2.3. Génération du dessin à l'écran

Pour dessiner les différentes formes géométriques à l'écran, il existe des algorithmes de tracé pour chaque forme de base. Ceux-ci permettent de générer une approximation de la forme désirée sur la base de ses paramètres. Le détail de chacun de ces algorithmes est décrit dans la partie 2.

Exemple de dessin généré :

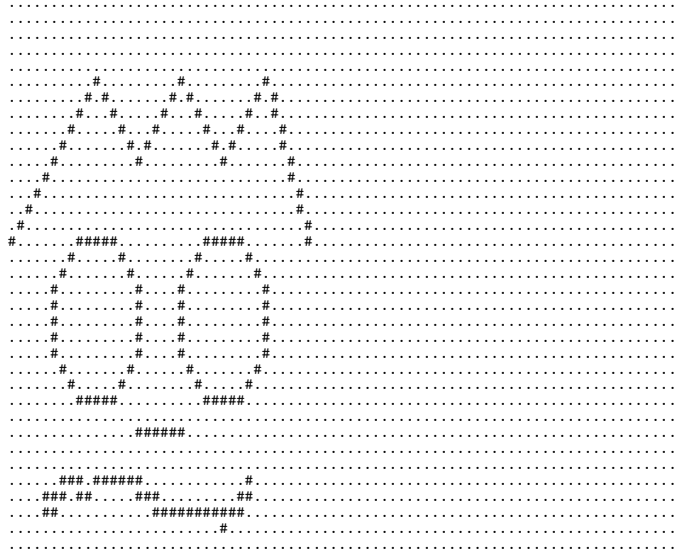
```

Veillez choisir une action :
A- Ajouter une forme
B- Afficher la liste des formes
C- Supprimer une forme
D- Tracer le dessin
E- Aide
[Autres actions]
>> Votre choix : B
    Liste des formes :
        1 : CIRCLE 20 10 5
        2 : CIRCLE 20 25 5
    
```

```

3 : LINE 5 20 10 25
4 : POLYGON 15 0 5 10 10 15 5 20 10 25 5 30 15 35
5 : CURVE 35 5 25 5 40 30 30 30
>> Votre choix : D

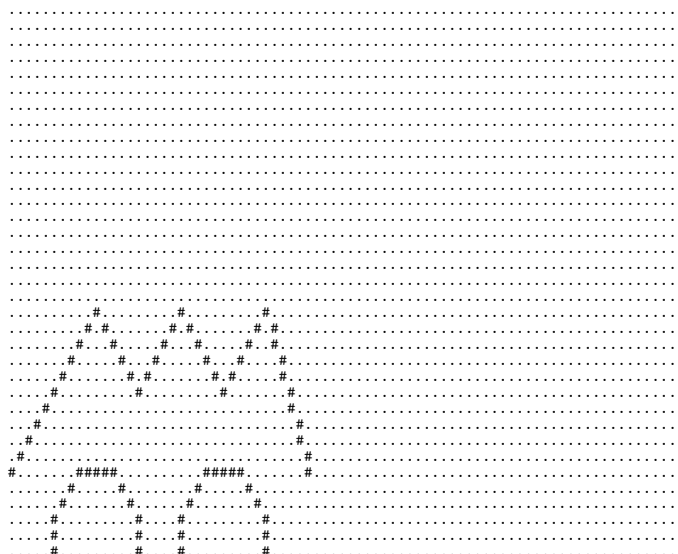
```



Il est à noter que les formes géométriques créées à partir de coordonnées se retrouvant partiellement ou totalement à l'extérieur de la zone de dessin sont acceptées. Celles-ci seront stockées en mémoire mais n'apparaîtront pas à l'écran.

Dans le cas où seulement une partie de la forme est à l'extérieur de la zone de dessin, il est nécessaire d'afficher à l'écran la portion du dessin se retrouvant à l'intérieur de la zone de dessin.

Exemple : Même figure dessinée avec des coordonnées différentes



Première partie

Classes de base des formes

Pour débiter ce travail, nous allons d'abord développer la partie du code de l'application permettant de créer et de stocker des formes spécifiques : Point, Line, Circle, Square, Rectangle et Polygon. On ne s'intéressera donc pas à la notion de calques, de fenêtres ou du dessin des formes à l'écran pour le moment. Comme nous le verrons plus en détails à la section 4, nous allons créer une classe abstraite (générique) pour la gestion de ces formes. Cette classe générique sera indépendante d'une forme particulière afin de faciliter les développements qui suivront dans la deuxième partie.

Dans cette partie nous allons créer plusieurs fichiers d'en-tête (.h) et de code source (.cpp). Nous utiliserons l'héritage et le polymorphisme pour gérer différents types de formes de manière uniforme.

3. Classe Point

Un point est une forme graphique qui représente une position dans l'espace. Notre application de dessin vectoriel étant textuelle nous utiliserons un caractère de remplissage pour le représenter dans notre espace à 2 dimensions. Dans la suite du document, nous prendrons comme caractère de remplissage le '#'.

La représentation en mémoire d'un point se fera par une classe `Point` composée de deux attributs privés `posX` et `posY` représentant ses coordonnées sur les axes x et y.

```
// Point.h
#ifndef POINT_H
#define POINT_H

#include <iostream>

class Point {
private:
    int posX;
    int posY;

public:
    // Constructeurs
    Point();
    Point(int x, int y);
    Point(const Point& other);

    // Destructeur
    ~Point();

    // Getters
    int getPosX() const { return posX; }
    int getPosY() const { return posY; }

    // Setters
    void setPosX(int x) { posX = x; }
```

```
void setPosY(int y) { posY = y; }

// Méthodes
void print() const;

// Opérateurs
Point& operator=(const Point& other);
bool operator==(const Point& other) const;
};

#endif // POINT_H
```

Pour manipuler un point, les méthodes suivantes doivent être définies dans le fichier Point.cpp :

```
// Point.cpp
#include "Point.h"

Point::Point() : posX(0), posY(0) {}

Point::Point(int x, int y) : posX(x), posY(y) {}

Point::Point(const Point& other) : posX(other.posX), posY(other.posY) {}

Point::~Point() {
    // Rien à libérer car pas d'allocation dynamique
}

void Point::print() const {
    std::cout << "POINT " << posX << " " << posY << std::endl;
}

Point& Point::operator=(const Point& other) {
    if (this != &other) {
        posX = other.posX;
        posY = other.posY;
    }
    return *this;
}

bool Point::operator==(const Point& other) const {
    return (posX == other.posX && posY == other.posY);
}
```

Où :

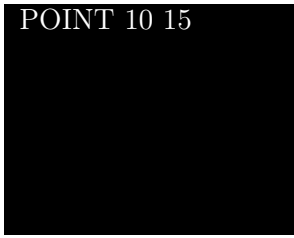
- Le constructeur par défaut Point() initialise un point à l'origine (0,0).
- Le constructeur paramétré Point(int x, int y) permet de créer un point avec des coordonnées spécifiques.
- Le constructeur de copie Point(const Point& other) permet de créer une copie d'un point existant.
- Le destructeur ~Point() libère les ressources (ici, aucune ressource dynamique à libérer).
- La méthode void print() const permet d'afficher à l'écran les informations d'un Point sous la forme suivante : POINT posX posY

Exemple d'utilisation

```
Point p(10, 15);
p.print();
// Pas besoin de delete car
// p est sur la pile
```

Résultat à l'écran

POINT 10 15



4. Classe abstraite Shape

Nous avons maintenant la possibilité de créer des points. Cependant le système de calque présenté dans la partie 1.1 à la page 5 doit permettre de regrouper des formes de type différent. De plus, chaque forme doit pouvoir être identifiée de manière unique dans l'application. Ce numéro doit être indépendant de la forme mais aussi du calque auquel la forme appartient car l'utilisateur doit avoir la possibilité de faire basculer une forme d'un calque à un autre.

L'application doit donc être en mesure de manipuler une classe abstraite permettant de représenter un type de forme quelconque : *Point*, *Line*, *Circle*, etc. C'est ce qu'on appelle la **généricité** en programmation. En C++ cette généricité est obtenue avec le polymorphisme et l'héritage.

La classe abstraite **Shape** servira de base pour toutes les formes géométriques. Elle définit une interface commune que toutes les formes devront implémenter.

```
// Shape.h
#ifndef SHAPE_H
#define SHAPE_H

#include <iostream>
#include <vector>
#include "Point.h"

// Déclaration anticipée
class Pixel;

class Shape {
protected:
    int id;
    static int nextId;

public:
    // Constructeur
    Shape();

    // Destructeur virtuel
    virtual ~Shape() = default;

    // Méthodes virtuelles pures (classe abstraite)
    virtual void print() const = 0;
    virtual std::vector<Pixel> toPixels() const = 0;
```

```
// Getter
int getId() const { return id; }
};

#endif // SHAPE_H

// Shape.cpp
#include "Shape.h"

// Initialisation du membre statique
int Shape::nextId = 1;

Shape::Shape() : id(nextId++) {
    // L'ID est automatiquement incrémenté
}
```

Note : Cette classe abstraite donne la possibilité d'ajouter une nouvelle forme à notre application en limitant l'impact sur le code déjà écrit. Ce type d'avantage correspond à l'indicateur appelé maintenabilité pour la [qualité logiciel](#).

4.1. Gestion des identifiants uniques

Les formes sont identifiées par un numéro unique. Pour garantir cette unicité, la méthode à utiliser est la suivante :

1. Créer un attribut statique **nextId** initialisé à 1 dans la classe **Shape**
2. Incrémenter la valeur de cet attribut à chaque création d'une forme géométrique dans le constructeur de **Shape**.

Comme montré dans le code ci-dessus, l'attribut statique **nextId** est partagé entre toutes les instances de **Shape** et de ses classes dérivées. Ainsi, chaque nouvelle forme reçoit automatiquement un ID unique lors de sa construction.

5. Classe Line

Une ligne est une forme géométrique correspondant à un segment délimité par deux points. Elle hérite de la classe **Shape**.

```
// Line.h
#ifndef LINE_H
#define LINE_H

#include "Shape.h"

class Line : public Shape {
private:
    Point p1;
    Point p2;

public:
```

```
// Constructeurs
Line(const Point& point1, const Point& point2);
Line(int x1, int y1, int x2, int y2);

// Destructeur
~Line() override = default;

// Getters
const Point& getP1() const { return p1; }
const Point& getP2() const { return p2; }

// Setters
void setP1(const Point& point) { p1 = point; }
void setP2(const Point& point) { p2 = point; }

// Méthodes override (implémentation des méthodes virtuelles pures)
void print() const override;
std::vector<Pixel> toPixels() const override;
};

#endif // LINE_H
```

Les méthodes associées à cette classe sont les suivantes :

```
// Line.cpp
#include "Line.h"
#include "Pixel.h"
#include <cmath>

Line::Line(const Point& point1, const Point& point2)
    : Shape(), p1(point1), p2(point2) {}

Line::Line(int x1, int y1, int x2, int y2)
    : Shape(), p1(x1, y1), p2(x2, y2) {}

void Line::print() const {
    std::cout << "LINE " << p1.getPosX() << " " << p1.getPosY()
                << " " << p2.getPosX() << " " << p2.getPosY()
                << std::endl;
}

std::vector<Pixel> Line::toPixels() const {
    // Implémentation de l'algorithme de Bresenham
    // (sera détaillée dans la partie 2)
    std::vector<Pixel> pixels;
    // ... code de l'algorithme ...
    return pixels;
}
```

Où :

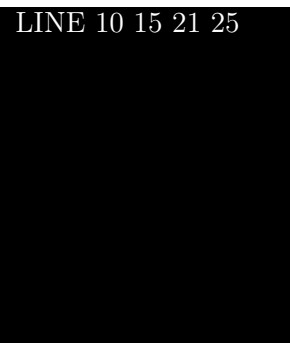
- Le constructeur `Line(const Point& point1, const Point& point2)` permet de créer un segment à partir de deux points donnés en paramètres.

- Le constructeur `Line(int x1, int y1, int x2, int y2)` permet de créer un segment à partir de 4 coordonnées.
- La méthode `void print() const` affiche les informations du segment : `LINE x1 y1 x2 y2`
- La méthode `std::vector<Pixel> toPixels() const` génère l'ensemble des pixels constituant le segment (détaillé en partie 2).

Exemple d'utilisation

```
Point p1(10, 15);
Point p2(21, 25);
Line l(p1, p2);
l.print();
// Ou directement:
Line l2(10, 15, 21, 25);
l2.print();
```

Résultat à l'écran



6. Classe Circle

La forme cercle est représentée par un point centre et un rayon. Elle hérite de la classe `Shape`.

```
// Circle.h
#ifndef CIRCLE_H
#define CIRCLE_H

#include "Shape.h"

class Circle : public Shape {
private:
    Point center;
    int radius;

public:
    // Constructeurs
    Circle(const Point& c, int r);
    Circle(int x, int y, int r);

    // Destructeur
    ~Circle() override = default;

    // Getters
    const Point& getCenter() const { return center; }
    int getRadius() const { return radius; }

    // Setters
    void setCenter(const Point& c) { center = c; }
```

```
void setRadius(int r) { radius = r; }

// Méthodes override
void print() const override;
std::vector<Pixel> toPixels() const override;
};

#endif // CIRCLE_H
```

Tout comme pour les classes précédentes, il est nécessaire d'implémenter les méthodes pour la manipulation de ce type :

```
// Circle.cpp
#include "Circle.h"
#include "Pixel.h"

Circle::Circle(const Point& c, int r)
    : Shape(), center(c), radius(r) {}

Circle::Circle(int x, int y, int r)
    : Shape(), center(x, y), radius(r) {}

void Circle::print() const {
    std::cout << "CIRCLE " << center.getPosX() << " "
                << center.getPosY() << " " << radius << std::endl;
}

std::vector<Pixel> Circle::toPixels() const {
    // Implémentation de l'algorithme d'Andres/Bresenham
    // (sera détaillée dans la partie 2)
    std::vector<Pixel> pixels;
    // ... code de l'algorithme ...
    return pixels;
}
```

7. Classe Square

La forme carré est représentée par un point (coin supérieur gauche) et une longueur. Les positions des 3 autres coins peuvent être calculées comme suit :

```

. . . . .
(px,py). . . (px,py+length)
. . # # # . .
. . # . . # . .
. . # . . # . .
. . # # # . .
(px+length,py). . . (px+length,py+length)
. . . . .
```

```
// Square.h
#ifndef SQUARE_H
#define SQUARE_H

#include "Shape.h"

class Square : public Shape {
private:
    Point topLeft;
    int length;

public:
    // Constructeurs
    Square(const Point& tl, int len);
    Square(int x, int y, int len);

    // Destructeur
    ~Square() override = default;

    // Getters
    const Point& getTopLeft() const { return topLeft; }
    int getLength() const { return length; }

    // Setters
    void setTopLeft(const Point& tl) { topLeft = tl; }
    void setLength(int len) { length = len; }

    // Méthodes override
    void print() const override;
    std::vector<Pixel> toPixels() const override;
};

#endif // SQUARE_H
```

8. Classe Rectangle

La forme rectangle est représentée par un point (coin supérieur gauche), une largeur et une hauteur. Les positions des 3 autres coins peuvent être calculées comme suit :

```
. . . . .
(px,py). . . . .(px,py+length)
. . # # # # # . .
. . # . . . . # . .
. . # . . . . # . .
. . # # # # # # # . .
(px+width,py). . . . . (px+width,py+length)
. . . . .
```

```
// Rectangle.h
#ifndef RECTANGLE_H
```

```
#define RECTANGLE_H

#include "Shape.h"

class Rectangle : public Shape {
private:
    Point topLeft;
    int width;
    int height;

public:
    // Constructeurs
    Rectangle(const Point& tl, int w, int h);
    Rectangle(int x, int y, int w, int h);

    // Destructeur
    ~Rectangle() override = default;

    // Getters
    const Point& getTopLeft() const { return topLeft; }
    int getWidth() const { return width; }
    int getHeight() const { return height; }

    // Setters
    void setTopLeft(const Point& tl) { topLeft = tl; }
    void setWidth(int w) { width = w; }
    void setHeight(int h) { height = h; }

    // Méthodes override
    void print() const override;
    std::vector<Pixel> toPixels() const override;
};

#endif // RECTANGLE_H
```

9. Classe Polygon

Le polygone est représenté comme un ensemble de points à relier. La classe utilise un `std::vector` pour stocker les points de manière dynamique.

NB : Pour avoir un polygone fermé, il est nécessaire que les coordonnées de son premier point soient les mêmes que celles du dernier point.

```
// Polygon.h
#ifndef POLYGON_H
#define POLYGON_H

#include "Shape.h"
#include <vector>

class Polygon : public Shape {
```

```
private:
    std::vector<Point> points;

public:
    // Constructeurs
    Polygon();
    Polygon(const std::vector<Point>& pts);
    Polygon(const std::vector<int>& coords); // coords: x1,y1,x2,y2,...

    // Destructeur
    ~Polygon() override = default;

    // Getters
    const std::vector<Point>& getPoints() const { return points; }
    int getNumPoints() const { return points.size(); }

    // Méthodes
    void addPoint(const Point& p);
    void addPoint(int x, int y);

    // Méthodes override
    void print() const override;
    std::vector<Pixel> toPixels() const override;
};

#endif // POLYGON_H

// Polygon.cpp
#include "Polygon.h"
#include "Line.h"

Polygon::Polygon() : Shape() {}

Polygon::Polygon(const std::vector<Point>& pts)
    : Shape(), points(pts) {}

Polygon::Polygon(const std::vector<int>& coords) : Shape() {
    if (coords.size() % 2 != 0) {
        std::cerr << "Erreur: nombre impair de coordonnées" << std::endl;
        return;
    }

    for (size_t i = 0; i + 1 < coords.size(); i += 2) {
        points.push_back(Point(coords[i], coords[i+1]));
    }
}

void Polygon::addPoint(const Point& p) {
    points.push_back(p);
}

void Polygon::addPoint(int x, int y) {
```

```

    points.push_back(Point(x, y));
}

void Polygon::print() const {
    std::cout << "POLYGON";
    for (const auto& p : points) {
        std::cout << " " << p.getPosX() << " " << p.getPosY();
    }
    std::cout << std::endl;
}

std::vector<Pixel> Polygon::toPixels() const {
    std::vector<Pixel> pixels;

    if (points.size() < 2) return pixels;

    // Relier chaque paire de points consécutifs
    for (size_t i = 0; i < points.size() - 1; ++i) {
        Line line(points[i], points[i+1]);
        auto linePixels = line.toPixels();
        pixels.insert(pixels.end(), linePixels.begin(), linePixels.end());
    }

    return pixels;
}

```

Deuxième partie

Affichage de l'image à l'écran

10. Classes pour l'affichage

10.1. Classe Pixel

Pour chaque forme que nous avons créée dans la partie précédente il nous faut la transformer en pixels et l'afficher à l'écran. C'est à dire calculer les points à remplir dans la matrice 2D.

Pour réaliser cette partie nous proposons de procéder de la manière suivante :

1. Initialiser la matrice 2D à 0
2. Pour chaque forme :
 - a) Calculer les positions (coordonnées des cases dans la matrice) des pixels à remplir selon le type de la forme
 - b) Remplacer les 0 par des 1 dans la matrice aux positions retournées dans l'étape précédente.
3. Parcourir la matrice, pour chaque case :
 - si la case = 0 alors afficher '.'
 - si la case = 1 alors afficher '#'

Pour ce faire, il faut définir une classe `Pixel` :

```
// Pixel.h
#ifndef PIXEL_H
#define PIXEL_H

class Pixel {
private:
    int x;
    int y;

public:
    // Constructeurs
    Pixel();
    Pixel(int px, int py);
    Pixel(const Pixel& other);

    // Destructeur
    ~Pixel();

    // Getters
    int getX() const { return x; }
    int getY() const { return y; }

    // Setters
    void setX(int px) { x = px; }
    void setY(int py) { y = py; }
```

```
// Opérateurs
Pixel& operator=(const Pixel& other);
bool operator==(const Pixel& other) const;
};

#endif // PIXEL_H

// Pixel.cpp
#include "Pixel.h"

Pixel::Pixel() : x(0), y(0) {}

Pixel::Pixel(int px, int py) : x(px), y(py) {}

Pixel::Pixel(const Pixel& other) : x(other.x), y(other.y) {}

Pixel::~Pixel() {}

Pixel& Pixel::operator=(const Pixel& other) {
    if (this != &other) {
        x = other.x;
        y = other.y;
    }
    return *this;
}

bool Pixel::operator==(const Pixel& other) const {
    return (x == other.x && y == other.y);
}
```

Où :

- Le constructeur `Pixel(int px, int py)` permet de créer un pixel avec des coordonnées spécifiques.
- Le destructeur `~Pixel()` libère les ressources (aucune allocation dynamique ici).

10.2. Classe Area

L'écran d'un ordinateur est composé de pixels, l'origine (0,0) se situe en haut à gauche de l'écran. Dans ce projet nous souhaitons garder cette disposition. C'est à dire l'axe *X* est sur la verticale et l'axe *Y* et sur l'horizontale, comme le montre la figure 6.

Cette zone de dessin est vue comme étant une matrice (un tableau à 2D) où chaque case est considérée comme un pixel. Cette structure doit contenir tous les pixels à afficher ainsi que leurs états (vide, rempli). La structure doit contenir aussi toutes les formes (collection de pointeurs intelligents sur `Shape`) qui sont dessinées dans cette zone de dessin.

```
// Area.h
#ifndef AREA_H
#define AREA_H
```

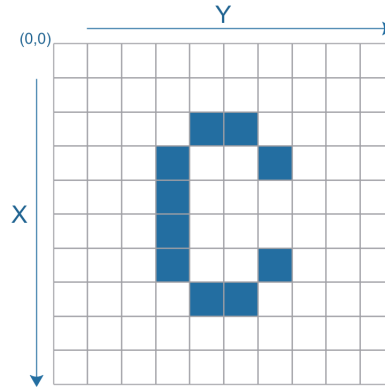



FIGURE 6 – Écran

```
#include <vector>
#include <memory>
#include "Shape.h"

class Area {
private:
    unsigned int width;
    unsigned int height;
    std::vector<std::vector<bool>> mat;
    std::vector<std::shared_ptr<Shape>> shapes;

public:
    // Constructeur
    Area(unsigned int w, unsigned int h);

    // Destructeur
    ~Area() = default;

    // Getters
    unsigned int getWidth() const { return width; }
    unsigned int getHeight() const { return height; }
    const std::vector<std::shared_ptr<Shape>>& getShapes() const {
        return shapes;
    }

    // Méthodes de gestion des formes
    void addShape(std::shared_ptr<Shape> shape);
    void removeShape(int id);
    std::shared_ptr<Shape> getShape(int id) const;

    // Méthodes de dessin
    void clear();
    void draw();
    void print() const;
    void erase();
};
```

```
#endif // AREA_H
```

Les deux variables `width` et `height` représentent la taille de la zone d'affichage réel à l'écran. La matrice `mat` est un tableau 2D de `bool`, chaque valeur peut contenir soit `false` pour designer un pixel vide ou `true` pour dire que le pixel est plein et noir.

Pour gérer la zone de dessin, il est demandé de définir les méthodes suivantes :

```
// Area.cpp
#include "Area.h"
#include <iostream>
#include <algorithm>

Area::Area(unsigned int w, unsigned int h)
    : width(w), height(h), mat(h, std::vector<bool>(w, false)) {}

void Area::addShape(std::shared_ptr<Shape> shape) {
    shapes.push_back(shape);
}

void Area::removeShape(int id) {
    shapes.erase(
        std::remove_if(shapes.begin(), shapes.end(),
            [id](const std::shared_ptr<Shape>& s) {
                return s->getId() == id;
            }),
        shapes.end()
    );
}

std::shared_ptr<Shape> Area::getShape(int id) const {
    auto it = std::find_if(shapes.begin(), shapes.end(),
        [id](const std::shared_ptr<Shape>& s) {
            return s->getId() == id;
        });
    return (it != shapes.end()) ? *it : nullptr;
}

void Area::clear() {
    for (auto& row : mat) {
        std::fill(row.begin(), row.end(), false);
    }
}

void Area::draw() {
    clear();

    for (const auto& shape : shapes) {
        auto pixels = shape->toPixels();
        for (const auto& pixel : pixels) {
```

```

    int x = pixel.getX();
    int y = pixel.getY();

    // Vérifier que le pixel est dans les limites
    if (x >= 0 && x < static_cast<int>(height) &&
        y >= 0 && y < static_cast<int>(width)) {
        mat[x][y] = true;
    }
}
}

void Area::print() const {
    for (const auto& row : mat) {
        for (bool pixel : row) {
            std::cout << (pixel ? '#' : '.');
        }
        std::cout << std::endl;
    }
}

void Area::erase() {
    shapes.clear();
    clear();
}

```

Où :

- Le constructeur `Area(unsigned int w, unsigned int h)` permet de créer une zone et alloue la mémoire nécessaire pour stocker le tableau des pixels à afficher à l'écran.
- La méthode `void addShape(std::shared_ptr<Shape> shape)` permet d'ajouter une forme à la zone de dessin en l'insérant dans le vecteur des formes.
- La méthode `void clear()` permet d'initialiser tous les pixels à `false`.
- La méthode `void erase()` permet de supprimer toutes les formes dessinées dans cette zone de dessin. Les smart pointers libèrent automatiquement la mémoire.
- La méthode `void draw()` permet de dessiner toutes les formes dans la zone de dessin. Son objectif est de parcourir toutes les formes. Puis, pour chaque forme, elle appelle la méthode `toPixels()` qui retourne la liste des pixels à colorier. Enfin, cette liste de pixels est parcourue et les valeurs à `true` sont affectées aux bons pixels dans la matrice `mat`.
- La méthode `void print() const` permet d'afficher à l'écran les pixels de la matrice `mat` à l'aide de `std::cout`. L'affichage se fait en utilisant des caractères ASCII : `'.'` pour un pixel vide et `'#'` pour un pixel rempli.

Exemple d'utilisation :

```

Area drawZone(12, 14);
auto shape1 = std::make_shared<Line>(5, 5, 10, 10);
auto shape2 = std::make_shared<Circle>(5, 5, 4);

```

```
drawZone.addShape(shape1);
drawZone.addShape(shape2);
drawZone.draw();
drawZone.print();
```

```
drawZone.erase();
drawZone.draw();
drawZone.print();
```

1er appel a draw

```
.....
...#####....
..##...##...
.##.....##...
.#.....#...
.#...#...#...
.#...#...#...
.##...###...
..##...##...
...#####.#...
.....#...
.....
```

2eme appel a draw

```
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
```

11. Tracer les formes

Afin de calculer la position des pixels à remplir dans la matrice, nous allons utiliser des algorithmes spécialisés pour chaque type de forme. Ces algorithmes vont nous retourner la liste des pixels à mettre à `true`. L'idée est d'implémenter la méthode virtuelle pure :

```
virtual std::vector<Pixel> toPixels() const = 0;
```

pour chaque classe dérivée de `Shape`.

11.1. Tracer un point

La méthode `toPixels()` pour un point est la plus simple. Elle retourne un vecteur contenant un seul pixel correspondant à la position du point.

```
// Dans Point.h - ajouter l'héritage de Shape
class Point : public Shape {
private:
    int posX;
    int posY;

public:
    // ... constructeurs existants ...
```

```
// Méthodes override
void print() const override;
std::vector<Pixel> toPixels() const override;
};

// Dans Point.cpp
std::vector<Pixel> Point::toPixels() const {
    std::vector<Pixel> pixels;
    pixels.push_back(Pixel(posX, posY));
    return pixels;
}
```

11.2. Tracer une ligne

La méthode `toPixels()` pour une ligne permet de calculer la position des pixels à remplir pour tracer une ligne sans trous. Par exemple la forme LINE 2 2 12 18 permettra de tracer la ligne suivante :

```
.....
.....
..#.....
...##.....
....#.....
.....##.....
.....##.....
.....#.....
.....##.....
.....#.....
.....##.....
.....##.....
.....#.....
```

Pour calculer la position des pixels qu'occupe une ligne, on pourrait utiliser la fonction :

$$f(x) = ax + b$$

Cependant, l'écran d'un ordinateur est discret ($\neq$ continu), utiliser directement cette méthode peut engendrer des lignes avec des trous notamment lorsque la ligne est diagonale. Pour tracer une ligne sans trous sur un écran discret, il existe de nombreux [algorithmes de tracé de segment](#) dont l'algorithme de [Bresenham](#). Afin de simplifier le tracé, nous allons utiliser l'algorithme de Nicolas Flasque (Enseignant à Efrei Paris) qui se base sur une méthode utilisant uniquement des nombres entiers et non pas des flottants.

L'idée globale est que pour calculer la position des pixels que va occuper une ligne, nous prenons en paramètres les coordonnées de début $(x1, y1)$ et les coordonnées de fin $(x2, y2)$. Les pixels de la ligne sont calculés en effectuant des décalages entre les deux positions début et fin : (dx, dy) .

Implémentation en C++ :

Data: x_1, y_1, x_2, y_2
Result: *pixels* (vecteur de Pixel)
 $dx \leftarrow |x_2 - x_1|;$
 $dy \leftarrow |y_2 - y_1|;$
 $sx \leftarrow 1;$
 $sy \leftarrow 1;$
if $x_1 > x_2$ **then**
 $sx \leftarrow -1;$
end
if $y_1 > y_2$ **then**
 $sy \leftarrow -1;$
end
 $delta \leftarrow dx - dy;$
while $(x_1 \neq x_2) \vee (y_1 \neq y_2)$ **do**
 ajouter Pixel(x_1, y_1) dans pixels;
 $e \leftarrow 2 * delta;$
 if $e > -dy$ **then**
 $delta \leftarrow delta - dy;$
 $x_1 \leftarrow x_1 + sx;$
 else
 if $e < dx$ **then**
 $delta \leftarrow delta + dx;$
 $y_1 \leftarrow y_1 + sy;$
 end
 end
end
ajouter Pixel(x_2, y_2) dans pixels;
retourner pixels;

Algorithm 1: Algorithme de tracé de ligne

```
// Dans Line.cpp
std::vector<Pixel> Line::toPixels() const {
    std::vector<Pixel> pixels;

    int x1 = p1.getPosX();
    int y1 = p1.getPosY();
    int x2 = p2.getPosX();
    int y2 = p2.getPosY();

    int dx = std::abs(x2 - x1);
    int dy = std::abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;
    int delta = dx - dy;

    while (x1 != x2 || y1 != y2) {
        pixels.push_back(Pixel(x1, y1));

        int e = 2 * delta;
        if (e > -dy) {
```

```

        delta -= dy;
        x1 += sx;
    }
    if (e < dx) {
        delta += dx;
        y1 += sy;
    }
}

pixels.push_back(Pixel(x2, y2));
return pixels;
}

```

11.3. Tracer un cercle

La méthode `toPixels()` pour un cercle permet de calculer la position des pixels à remplir pour tracer un cercle. Par exemple la forme `CIRCLE 4 4 3` permettra de tracer le cercle suivant :

```

.....
...###...
..#...#...
.#....#...
.#....#...
.#....#...
..#...#...
...###...
.....

```

Pour calculer tous les pixels à générer pour dessiner un cercle, il est demandé d'utiliser [l'algorithme de tracé de cercle d'Andres](#). Cet algorithme est aussi connu sous le nom de [Algorithme de tracé d'arc de cercle de Bresenham](#). Il est plus efficace que d'autres algorithmes de tracé de cercles, car il utilise uniquement des opérations entières (pas de virgules flottantes) avec moins de calculs.

Pour tracer un cercle, l'idée est de générer les pixels d'un seul octant (un huitième $\frac{1}{8}$), et de déduire le reste par symétrie comme illustré dans la figure 7. Grâce à cela, le nombre de calculs à faire est réduit.

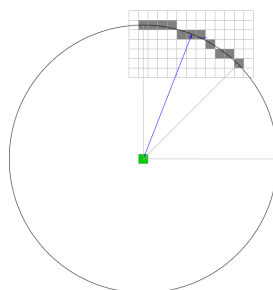


FIGURE 7 – Cercle dans un seul octant

L'algorithme calcule les points du cercle en utilisant uniquement des valeurs entières. Les étapes suivies sont :

Pour un cercle de centre (cx, cy) et de rayon r :

1. Initialiser deux variables x et y (coordonnées du cercle) à $(0, r)$;
2. Initialiser l'erreur de tracé d à $(1 - r)$;
3. Répéter jusqu'à ce que x dépasse y :
 - a) Dessiner huit points symétriques autour du centre du cercle. Les coordonnées de ces 8 points sont :
 - Point1 $(cx + x, cy + y)$
 - Point2 $(cx + y, cy + x)$
 - Point3 $(cx - x, cy + y)$
 - Point4 $(cx - y, cy + x)$
 - Point5 $(cx + x, cy - y)$
 - Point6 $(cx + y, cy - x)$
 - Point7 $(cx - x, cy - y)$
 - Point8 $(cx - y, cy - x)$
 - b) Si l'erreur de tracé d est négative, on incrémente x et on met à jour l'erreur de tracé en ajoutant $2x + 1$.
 - c) Sinon, on décrémente y , on incrémente x et on met à jour l'erreur de tracé en ajoutant $2x - 2y + 2$.

Implémentation en C++ :

```
// Dans Circle.cpp
std::vector<Pixel> Circle::toPixels() const {
    std::vector<Pixel> pixels;

    int x = 0;
    int y = radius;
    int d = radius - 1;

    int cx = center.getPosX();
    int cy = center.getPosY();

    while (y >= x) {
        // 8 octants
        pixels.push_back(Pixel(cx + x, cy + y));
        pixels.push_back(Pixel(cx + y, cy + x));
        pixels.push_back(Pixel(cx - x, cy + y));
        pixels.push_back(Pixel(cx - y, cy + x));
        pixels.push_back(Pixel(cx + x, cy - y));
        pixels.push_back(Pixel(cx + y, cy - x));
        pixels.push_back(Pixel(cx - x, cy - y));
        pixels.push_back(Pixel(cx - y, cy - x));

        if (d >= 2 * x) {
            d -= 2 * x + 1;
            x++;
        } else if (d < 2 * (radius - y)) {
            d += 2 * y - 1;
            y--;
        } else {

```


Data: cercle (centre cx , cy et rayon r)

Result: pixels (vecteur de Pixel)

méthode toPixels() :

```

     $x \leftarrow 0$  ;
     $y \leftarrow r$  ;
     $d \leftarrow r - 1$  ;
    while  $y \geq x$  do
        // Ajouter les 8 points symétriques
        pixels.push_back(Pixel( $cx + x, cy + y$ ));
        pixels.push_back(Pixel( $cx + y, cy + x$ ));
        pixels.push_back(Pixel( $cx - x, cy + y$ ));
        pixels.push_back(Pixel( $cx - y, cy + x$ ));
        pixels.push_back(Pixel( $cx + x, cy - y$ ));
        pixels.push_back(Pixel( $cx + y, cy - x$ ));
        pixels.push_back(Pixel( $cx - x, cy - y$ ));
        pixels.push_back(Pixel( $cx - y, cy - x$ ));
        if  $d \geq 2x$  then
             $d \leftarrow d - (2x + 1)$  ;
             $x \leftarrow x + 1$  ;
        end
        else if  $d < 2(r - y)$  then
             $d \leftarrow d + (2y - 1)$  ;
             $y \leftarrow y - 1$  ;
        end
        else
             $d \leftarrow d + 2(y - x - 1)$  ;
             $y \leftarrow y - 1$  ;
             $x \leftarrow x + 1$  ;
        end
    end
    retourner pixels;
```

```

         $d += 2 * (y - x - 1)$ ;
         $y--$ ;
         $x++$ ;
    }
}

return pixels;
}
```

11.4. Tracer carré, rectangle et polygone

Pour calculer la position des pixels qu'occupe un carré, il suffit de calculer la position des pixels des 4 segments en utilisant le même procédé utilisé dans la méthode `toPixels()` de la classe `Line`.

Nous rappelons grâce au schéma ci-après la méthode de calcul des positions des 4 segments.

.

```

(px,py). . . .(px,py+length)
. . # # # # . .
. . # . . # . .
. . # . . # . .
. . # # # # . .
(px+length,py). . . .(px+length,py+length)
. . . . . . . .

```

```

// Dans Square.cpp
std::vector<Pixel> Square::toPixels() const {
    std::vector<Pixel> pixels;

    int x = topLeft.getPosX();
    int y = topLeft.getPosY();

    // 4 côtés du carré
    Line top(x, y, x, y + length);
    Line right(x, y + length, x + length, y + length);
    Line bottom(x + length, y + length, x + length, y);
    Line left(x + length, y, x, y);

    auto pixelsTop = top.toPixels();
    auto pixelsRight = right.toPixels();
    auto pixelsBottom = bottom.toPixels();
    auto pixelsLeft = left.toPixels();

    pixels.insert(pixels.end(), pixelsTop.begin(), pixelsTop.end());
    pixels.insert(pixels.end(), pixelsRight.begin(), pixelsRight.end());
    pixels.insert(pixels.end(), pixelsBottom.begin(), pixelsBottom.end());
    pixels.insert(pixels.end(), pixelsLeft.begin(), pixelsLeft.end());

    return pixels;
}

```

Pour calculer les pixels d'un rectangle, le calcul est identique au calcul précédent :

```

// Dans Rectangle.cpp
std::vector<Pixel> Rectangle::toPixels() const {
    std::vector<Pixel> pixels;

    int x = topLeft.getPosX();
    int y = topLeft.getPosY();

    // 4 côtés du rectangle
    Line top(x, y, x, y + width);
    Line right(x, y + width, x + height, y + width);
    Line bottom(x + height, y + width, x + height, y);
    Line left(x + height, y, x, y);

    auto pixelsTop = top.toPixels();
    auto pixelsRight = right.toPixels();
    auto pixelsBottom = bottom.toPixels();

```

```
auto pixelsLeft = left.toPixels();

pixels.insert(pixels.end(), pixelsTop.begin(), pixelsTop.end());
pixels.insert(pixels.end(), pixelsRight.begin(), pixelsRight.end());
pixels.insert(pixels.end(), pixelsBottom.begin(), pixelsBottom.end());
pixels.insert(pixels.end(), pixelsLeft.begin(), pixelsLeft.end());

return pixels;
}
```

Pour les polygones la méthode est simple. Il suffit de relier l'ensemble des points par des segments (déjà implémenté dans Polygon.cpp).

Troisième partie

Gestion des commandes

Nous allons créer des commandes utilisateurs spécifiques permettant de manipuler les formes géométriques afin de les tracer dans la zone de dessin de l'application. Le rafraîchissement de l'affichage de la zone de dessin se fait de manière automatique après l'ajout d'une forme.

Celles-ci vont indiquer des ordres d'exécution à donner à l'application. Chaque commande doit être de la forme : **<nom commande> [paramètres]**. La présence des paramètres est optionnelle car cela dépend du type de l'ordre à effectuer.

Dans ce projet, la liste des commandes que le programme doit interpréter est la suivante :

- **clear** : effacer l'écran
- **exit** : quitter le programme
- **point x y** : ajouter un point
- **line x1 y1 x2 y2** : ajouter un segment reliant deux points (x1, y1) et (x2, y2)
- **circle x y radius** : ajouter un cercle de centre (x, y) et de rayon **radius**
- **square x y length** : ajouter un carré dont le coin supérieur gauche est (x, y) et de côté **length**.
- **rectangle x y width height** : ajouter un rectangle dont le coin supérieur gauche est (x, y), de largeur **width** et de longueur **height**
- **polygon x1 y1 x2 y2 x3 y3 ...** : ajouter un polygone avec la liste des points donnés
- **plot** : rafraîchir l'écran pour afficher toutes les formes géométriques de l'image
- **list** : afficher la liste de l'ensemble des formes géométriques qui composent l'image
- **delete id** : supprimer une forme à partir de son identifiant **id**.
- **erase** : supprimer toutes les formes d'une image.
- **help** : afficher la liste des commandes ainsi qu'un mini mode d'emploi

12. Les commandes utilisateur

Pour gérer la ligne de commande on va utiliser une boucle infinie qui va répéter ces étapes :

1. Afficher un prompt (par exemple ">").
2. Lire l'entrée de l'utilisateur en utilisant `std::getline()`.
3. Analyser la commande et remplir l'objet `Command`.
4. Vérifier si la commande est correcte.
5. Exécuter la commande de l'utilisateur.
6. Revenir à l'étape 1.

13. Classe Command

Les commandes utilisateur sont sous la forme suivante :

- Commencent toujours par une chaîne de caractères qui est le nom de la commande.
- Peut être suivie par :

- Des entiers
- Une autre chaîne suivie par des entiers ou d'autres chaînes

Une commande est donc un ensemble de chaînes de caractères suivi par un ensemble d'entiers. Voici quelques exemples :

```
>> line 10 12 30 45
>> delete 10
>> help
>> list
>> list layers
>> select layer 12
>> plot
```

Pour simplifier l'analyse de la ligne de commande, on va lire la chaîne saisie par l'utilisateur à l'aide de `std::getline()` puis on sépare cette chaîne en tokens en utilisant le caractère espace comme séparateur.

Chaque token sera placé dans la classe `Command`. Le premier token étant le nom de la commande va être stocké dans l'attribut `name`. Pour le reste, si le token est un entier, alors on le rajoute dans le vecteur `intParams`, si c'est une chaîne alors on la rajoute dans le vecteur `strParams`. Voir l'exemple à la figure 8.

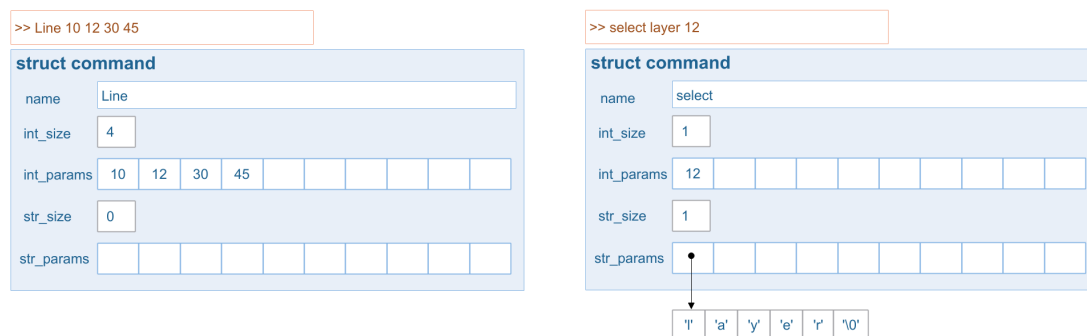


FIGURE 8 – Exemple de création d'un objet commande à partir d'une commande utilisateur saisie au clavier.

```
// Command.h
#ifndef COMMAND_H
#define COMMAND_H

#include <string>
#include <vector>
#include <memory>
#include "Area.h"

class Command {
private:
    std::string name;
    std::vector<int> intParams;
    std::vector<std::string> strParams;

public:
```

```

// Constructeur
Command();

// Destructeur
~Command() = default;

// Méthodes
void addIntParam(int p);
void addStrParam(const std::string& p);
void parse(const std::string& line);
bool execute(Area& area);
void clear();

// Getters
const std::string& getName() const { return name; }
const std::vector<int>& getIntParams() const { return intParams; }
const std::vector<std::string>& getStrParams() const { return strParams; }
};

#endif // COMMAND_H

// Command.cpp
#include "Command.h"
#include <sstream>
#include <iostream>
#include "Line.h"
#include "Circle.h"
#include "Square.h"
#include "Rectangle.h"
#include "Polygon.h"

Command::Command() {}

void Command::addIntParam(int p) {
    intParams.push_back(p);
}

void Command::addStrParam(const std::string& p) {
    strParams.push_back(p);
}

void Command::parse(const std::string& line) {
    clear();
    std::istringstream iss(line);
    std::string token;

    // Premier token = nom de la commande
    if (!(iss >> name)) return;

    // Traiter les autres tokens
    while (iss >> token) {
        try {

```

```

        int num = std::stoi(token);
        addIntParam(num);
    } catch (const std::invalid_argument&) {
        addStrParam(token);
    } catch (const std::out_of_range&) {
        std::cerr << "Nombre hors limites: " << token << std::endl;
    }
}

}

bool Command::execute(Area& area) {
    if (name == "point") {
        if (intParams.size() >= 2) {
            auto p = std::make_shared<Point>(intParams[0], intParams[1]);
            area.addShape(p);
            std::cout << "Point ajouté (ID: " << p->getId() << ")" << std::endl;
            return true;
        }
        std::cerr << "Usage: point x y" << std::endl;
    }
    else if (name == "line") {
        if (intParams.size() >= 4) {
            auto line = std::make_shared<Line>(
                intParams[0], intParams[1],
                intParams[2], intParams[3]
            );
            area.addShape(line);
            std::cout << "Ligne ajoutée (ID: " << line->getId() << ")" << std::endl;
            return true;
        }
        std::cerr << "Usage: line x1 y1 x2 y2" << std::endl;
    }
    else if (name == "circle") {
        if (intParams.size() >= 3) {
            auto circle = std::make_shared<Circle>(
                intParams[0], intParams[1], intParams[2]
            );
            area.addShape(circle);
            std::cout << "Cercle ajouté (ID: " << circle->getId() << ")" << std::endl;
            return true;
        }
        std::cerr << "Usage: circle x y radius" << std::endl;
    }
    else if (name == "square") {
        if (intParams.size() >= 3) {
            auto square = std::make_shared<Square>(
                intParams[0], intParams[1], intParams[2]
            );
            area.addShape(square);
            std::cout << "Carré ajouté (ID: " << square->getId() << ")" << std::endl;
            return true;
        }
    }
}

```

```

    }
    std::cerr << "Usage: square x y length" << std::endl;
}
else if (name == "rectangle") {
    if (intParams.size() >= 4) {
        auto rect = std::make_shared<Rectangle>(
            intParams[0], intParams[1],
            intParams[2], intParams[3]
        );
        area.addShape(rect);
        std::cout << "Rectangle ajouté (ID: "
            // Destructeur
~Layer() = default;

// Getters
int getId() const { return id; }
const std::string& getName() const { return name; }
bool isVisible() const { return visible; }
const std::vector<std::shared_ptr<Shape>>& getShapes() const {
    return shapes;
}

// Setters
void setName(const std::string& n) { name = n; }
void setVisible(bool v) { visible = v; }

// Méthodes
void addShape(std::shared_ptr<Shape> shape);
void removeShape(int shapeId);
void clear();
void print() const;
};
#endif // LAYER_H

// Layer.cpp
#include "Layer.h"
#include <iostream>
#include <algorithm>
int Layer::nextId = 1;
Layer::Layer() : id(nextId++), name("Layer_" + std::to_string(id)),
    visible(true) {}
Layer::Layer(const std::string& layerName)
    : id(nextId++), name(layerName), visible(true) {}
void Layer::addShape(std::shared_ptr<Shape> shape) {
    shapes.push_back(shape);
}
void Layer::removeShape(int shapeId) {
    shapes.erase(
        std::remove_if(shapes.begin(), shapes.end(),
            [shapeId](const std::shared_ptr<Shape>& s) {
                return s->getId() == shapeId;
            })
    ),

```



```
shapes.end()
);
}
void Layer::clear() {
    shapes.clear();
}
void Layer::print() const {
    std::cout << "Layer " << id << " (" << name << ") - "
    << (visible ? "Visible" : "Hidden") << " - "
    << shapes.size() << " forme(s)" << std::endl;
}
```

Pour gérer les calques, nous proposons d'ajouter les commandes suivantes :

```
list layers
    Affiche la liste des calques disponibles.
new layer [layer_name]
    Crée un nouveau calque.
select layer [ID]
    Sélectionne un calque.
delete layer [id]
    Supprime un calque.
set layer visible [id]
    ou set layer hide [id] : Affiche ou cache un calque.
```

14. Les conteneurs STL

Tout au long de ce projet nous avons utilisé des conteneurs STL comme `std::vector`. C++ offre d'autres conteneurs très utiles :

1. `std::vector` : tableau dynamique, accès direct en $\mathcal{O}(1)$
2. `std::list` : liste doublement chaînée, insertion/suppression en $\mathcal{O}(1)$ avec un itérateur
3. `std::map` : dictionnaire trié (arbre rouge-noir), opérations en $\mathcal{O}(\log n)$
4. `std::unordered_map` : table de hachage, accès moyen en $\mathcal{O}(1)$
5. `std::set` : ensemble trié sans doublons, opérations en $\mathcal{O}(\log n)$

Avantages des conteneurs STL :

- Gestion automatique de la mémoire
- Pas besoin de `realloc`
- Pas de taille limite fixe
- Algorithmes optimisés
- Sécurité (pas de débordement)

Exemple d'utilisation de `std::list` :

```
#include <list>
std::list<std::shared_ptr<Shape>> shapes;
// Insertion
shapes.push_back(shapePtr);
shapes.push_front(shapePtr);
// Parcours
```

```
for (auto& shape : shapes) {
    shape->print();
}
// Suppression
shapes.remove_if([id](const std::shared_ptr<Shape>& s) {
    return s->getId() == id;
});
```

15. Smart Pointers

Le C++ moderne utilise des smart pointers pour la gestion automatique de la mémoire :

15.1. std::unique_ptr

Propriété exclusive, pas de copie possible :

```
std::unique_ptr<Point> p = std::make_unique<Point>(10, 15);
// Pas besoin de delete, libération automatique
```

15.2. std::shared_ptr

Propriété partagée avec comptage de références :

```
std::shared_ptr<Shape> shape = std::make_shared<Circle>(5, 5, 3);
auto copy = shape; // Compteur de références incrémenté
// Libération quand le dernier shared_ptr est détruit
```

15.3. std::weak_ptr

Référence faible sans propriété (évite les références circulaires) :

```
std::shared_ptr<Layer> layer = std::make_shared<Layer>();
std::weak_ptr<Layer> weakRef = layer;
if (auto locked = weakRef.lock()) {
    // Utilisation temporaire sécurisée
    locked->print();
}
```

16. Compilation et Makefile

Pour compiler le projet, créez un Makefile :

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -Werror -pedantic
TARGET = pixel_tracer
SRCS = main.cpp Point.cpp Shape.cpp Line.cpp Circle.cpp
Square.cpp Rectangle.cpp Polygon.cpp Pixel.cpp
```

```
Area.cpp Command.cpp Layer.cpp Curve.cpp
OBJS = $(SRCS:.cpp=.o)
all: $(TARGET)
$(TARGET): $(OBJS)
$(CXX) $(CXXFLAGS) -o $@ $^
%.o: %.cpp
$(CXX) $(CXXFLAGS) -c $< -o $@
clean:
rm -f $(OBJS) $(TARGET)
run: $(TARGET)
./$(TARGET)
.PHONY: all clean run
```

Ou utilisez CMake (CMakeLists.txt) :

```
cmake_minimum_required(VERSION 3.10)
project(PixelTracer)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(pixel_tracer
main.cpp
Point.cpp
Shape.cpp
Line.cpp
Circle.cpp
Square.cpp
Rectangle.cpp
Polygon.cpp
Pixel.cpp
Area.cpp
Command.cpp
Layer.cpp
Curve.cpp
)
```